

Chapter 14: Methods

By Victoria Gonda

In the previous chapter, you learned about properties, which are constants and variables that are part of classes and objects. **Methods**, as you’ve already seen, are merely functions that reside inside a class or object.

In this chapter, you’ll take a closer look at methods. As with properties, you’ll begin to design more complex classes and objects. So, open the starter project and jump right in!

Method refresher

Consider `ArrayList.removeAt()`. It pops the item at a given index off an instance of an array list:

```
val numbers = arrayListOf(1, 2, 3)
numbers.removeAt(numbers.lastIndex)
println(numbers) // > [1, 2]
```



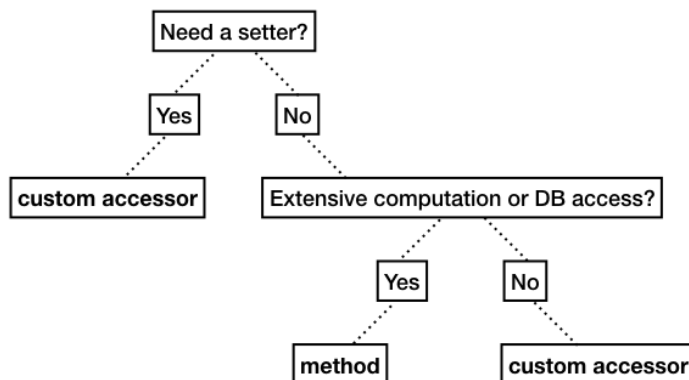
Methods like `removeAt()` help you control the data in the array list.

Comparing methods to getters and setters

With custom accessors, you saw in the last chapter that you could run code from inside a class within a property definition. That sounds a lot like a method. What's the difference? It really comes down to a matter of style, but there are a few helpful thoughts to help you decide.

Properties hold values that you can get and set, while methods perform work. Sometimes this distinction gets fuzzy when a method's sole purpose is to return a single value.

Should I implement this value getter
as a method or as a custom accessor?



Ask yourself whether you want to be able to set a value as well as get the value. A property can have a custom setter inside to write values. Another question to consider is whether the calculation requires extensive computation or reads from a database.

Even for a simple value, a method helps you indicate to future developers that the call is expensive in time and computational resources. If the call is cheap (as in constant time $O(1)$), stick with custom accessors.

Turning a function into a method

To explore methods, you will create a simple model for dates called `SimpleDate`. Be aware that the various Kotlin platforms, such as the JVM and JS, contain production-ready `Date` classes that correctly handle many of the subtle intricacies of dealing with dates and times.

Add this code so you have something to work with:

```
// 1
val months = arrayOf(
    "January", "February", "March",
    "April", "May", "June",
    "July", "August", "September",
    "October", "November", "December"
)

// 2
class SimpleDate1(var month: String)

// 3
fun monthsUntilWinterBreak(from: SimpleDate1): Int {
    return months.indexOf("December") - months.indexOf(from.month)
}
```

This creates:

1. An array of months.
2. A `SimpleDate1` class.
3. A method to calculate the months until winter break.

Note: If you live in the southern hemisphere, you can adjust the month for your winter break.

In the code above, how could you convert `monthsUntilWinterBreak()` into a method?

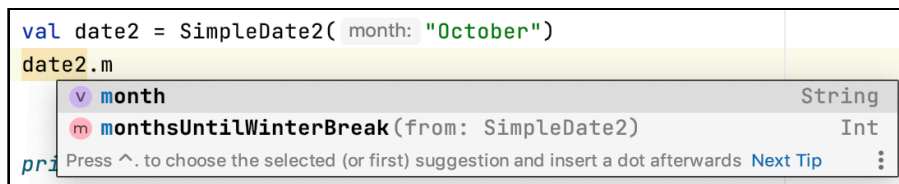
Create a new class, `SimpleDate2`, with `monthsUntilWinterBreak()` in the body of the class:

```
class SimpleDate2(var month: String) {  
    fun monthsUntilWinterBreak(from: SimpleDate2): Int {  
        return months.indexOf("December") -  
            months.indexOf(from.month)  
    }  
}
```

Making a method is as easy as moving the function inside the class definition.

There's no identifying keyword for a method; it really is just a function inside a class or object. You call methods on an instance using dot syntax just as you do for properties.

And just like properties, as soon as you start typing a method name, IntelliJ IDEA will provide suggestions. You can select one with the Up and Down arrow keys on your keyboard, and you can autocomplete the call by pressing Tab:



Try typing this into your file:

```
val date2 = SimpleDate2("October")  
println(date2.monthsUntilWinterBreak(date2)) // > 2
```

If you think about this code for a minute, you'll realize that the method's definition is awkward. There must be a way to access the content stored by the instance instead of passing the instance itself as a parameter to the method. It would be so much nicer to call this:

```
date.monthsUntilWinterBreak() // Error!
```

Introducing this

A class definition is like a blueprint, whereas an instance is a real object. To access the value of an instance, you use the keyword **this** inside the class.

The keyword **this** acts as a reference to the current instance. Make a `SimpleDate3` with this transformed method:

```
// 1
fun monthsUntilWinterBreak(): Int {
    // 2
    return months.indexOf("December") - months.indexOf(this.month)
}
```

Here's what changed:

1. Now there's no parameter in the method definition.
2. In the implementation, **this** replaces the old parameter name.

You can now call the method without passing a parameter. Try it out:

```
val date3 = SimpleDate3("September")
date3.monthsUntilWinterBreak() // 3
```

That's looking a lot cleaner! One more thing you can do to simplify the code is to remove **this**. That might be confusing because you just added it!

this is your reference to the instance, but most of the time you don't need to use it because Kotlin understands your intent if you just use a variable name.

While you can always use **this** to access the properties and methods of the current instance, most of the time you won't need to. In `monthsUntilWinterBreak()`, you can just say `month` instead of `this.month`:

```
return months.indexOf("December") - months.indexOf(month)
```

Most programmers use **this** only when it is required, for example, to disambiguate between a local variable and a property with the same name. You'll get more practice using **this** a little later.

Mini-exercise

Since `monthsUntilWinterBreak()` returns a single value and there's not much calculation involved, transform the method into a property with a custom getter.

Object methods

Like classes, Kotlin objects defined with the `object` keyword can have member functions that refer to the object itself.

For class companion objects, like companion object properties, you can use **companion object methods** to access data across all instances. You call companion object methods on the class itself, instead of on an instance. To define a companion object method, you put its definition inside the `companion object` block.

Object methods are useful for things that are *about* a type in general, rather than something about specific instances.

For an example, create this class:

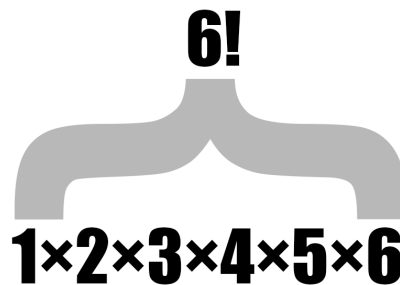
```
class MyMath {  
    // 1  
    companion object {  
        fun factorial(number: Int): Int {  
            // 2  
            return (1..number).fold(1) { a, b -> a * b }  
        }  
    }  
}  
  
// 3  
MyMath.factorial(6) // 720
```

Here, you use object methods to group similar methods into a class. In this case, math operations.

You might have custom calculations for things such as factorial. Instead of having a bunch of free-standing functions, you can group related functions together as methods in a class companion object. The class with its companion object is said to act as a **namespace**. If the class `MyMath` did not need any instances, you could instead just define `MyMath` as a Kotlin object to create the namespace.

Here's what's happening in MyMath:

1. You use the companion object block to declare the method on the class, which accepts an integer and returns an integer.
2. The implementation uses a higher-order function called `fold()`. It effectively follows the formula for calculating a factorial: “The product of all the whole numbers from 1 to n ”. You could write this using a `for` loop, but the higher-order function expresses your intent in a cleaner way.
3. You call the method on MyMath, rather than on an instance of the class.


$$6! \\ 1 \times 2 \times 3 \times 4 \times 5 \times 6$$

Methods gathered into an object or companion object will advantageously code complete in IntelliJ IDEA.

In this example, you can see all the math utility methods available to you by typing `MyMath.:`

```
val factorial = MyMath.  
  m factorial(number: Int) Int  
  m equals(other: Any?) Boolean
```

Mini-exercise

Add a method to the MyMath class that calculates the n -th triangle number. It will be similar to the factorial formula, except instead of multiplying the numbers, you add them.

For an example the triangle number for 3 is $1+2+3=6$. If you need a hint, you can look at the challenge solutions for this chapter.

Extension methods

Sometimes you want to add functionality to a class but don't want to muddy up the original definition. And sometimes you can't add the functionality because you don't have access to the source code. Just as for properties, it is possible to augment an existing class or object (even one you do not have the source code for) by adding methods to it.

Suppose you are using a `SimpleDate` class provided by a library that you don't have the source code for. The class provides a method to calculate months until winter break, but you'd like to have the ability to know the number of months until summer break.

To add an extension method onto a class, define a new function with the function name appended to the class name, like so:

```
fun SimpleDate.monthsUntilSummerBreak(): Int {  
    val monthIndex = months.indexOf(month)  
    return if (monthIndex in 0..months.indexOf("June")) {  
        months.indexOf("June") - monthIndex  
    } else if (monthIndex in  
        months.indexOf("June")..months.indexOf("August")) {  
        0  
    } else {  
        months.indexOf("June") + (12 - monthIndex)  
    }  
}
```

Note: If you live in the southern hemisphere, you can adjust the month for your summer break.

This creates an extension method `monthsUntilSummerBreak()` on the `SimpleDate` class.

You can use the extension method just like any other method call on an instance of the class:

```
val date = SimpleDate()  
date.month = "December"  
println(date.monthsUntilSummerBreak()) // > 6
```


You can add extension methods onto built-in types as well:

```
fun Int.abs(): Int {  
    return if (this < 0) -this else this  
}  
  
println(4.abs())    // > 4  
println((-4).abs()) // > 4
```

You're calling the extension method directly on a number, a literal value of the `Int` class.

Note: The Kotlin standard library has an `abs()` function that you would normally use.

Companion object extensions

If your class has a companion object, you can add extension methods to it by using the implicit companion object name `Companion`, or by using the custom name if the companion object has one.

As an example, you can add a method named `primeFactors()` to `MyMath` using a companion object extension:

```
fun MyMath.Companion.primeFactors(value: Int): List<Int> {  
    // 1  
    var remainingValue = value  
    // 2  
    var testFactor = 2  
    val primes = mutableListOf<Int>()  
    // 3  
    while (testFactor * testFactor <= remainingValue) {  
        if (remainingValue % testFactor == 0) {  
            primes.add(testFactor)  
            remainingValue /= testFactor  
        } else {  
            testFactor += 1  
        }  
    }  
  
    if (remainingValue > 1) {  
        primes.add(remainingValue)  
    }  
  
    return primes  
}
```

This method finds the prime factors for a given number. For example, 81 returns [3, 3, 3, 3]. Here's what's happening in the code:

1. The value passed in as a parameter is assigned to the mutable variable, `remainingValue`, so that it can be changed as the calculation runs.
2. The `testFactor` starts with a value of 2 and will be divided into `remainingValue`.
3. The logic runs a loop until the `remainingValue` is exhausted. If it divides evenly, meaning there's no remainder, that value of the `testFactor` is set aside as a prime factor. If it doesn't divide evenly, `testFactor` is incremented for the next loop.

This algorithm takes a brute-force approach, but does contain one optimization: the square of the `testFactor` should never be larger than the `remainingValue`. If it is, the `remainingValue` itself must be prime and is added to the primes list.

You've now added a method to `MyMath` without changing its original definition. Verify that the extension works with this code:

```
MyMath.primeFactors(81) // [3, 3, 3, 3]
```

Challenges

As you work through these challenges, remember you can look at the solutions for this chapter at any time for a hint or to check your work.

1. Given the `Circle` class below:

```
import kotlin.math.PI

class Circle(var radius: Double = 0.0) {
    val area: Double
    get() {
        return PI * radius * radius
    }
}
```

Write a method that can change an instance's area by a growth factor. For example if you call `circle.grow(factor = 3)`, the area of the instance will triple.

Hint: Make `area` a `var` and add a setter to it.

2. Here is a naïve way of writing `advance()` for the `SimpleDate` class you saw earlier in the chapter:

```
val months = arrayOf(
    "January", "February", "March",
    "April", "May", "June",
    "July", "August", "September",
    "October", "November", "December"
)

class SimpleDate(var month: String, var day: Int = 0) {
    fun advance() {
        day += 1
    }
}

var date = SimpleDate(month = "December", day = 31)
date.advance()
date.month // December; should be January!
date.day // 32; should be 1!
```

What happens when the function should go from the end of one month to the start of the next? Rewrite `advance()` to account for advancing from December 31st to January 1st.

3. Create a Kotlin object named `MyMath` with `isEven()` and `isOdd()` methods that return `true` if a number is even or odd respectively.
4. Add extension methods `isEven()` and `isOdd()` to `Int`.

Note: Generally, you want to be careful about what functionality you add to standard library types as it can cause confusion for readers.

5. Add the extension method `primeFactors()` to `Int`. Since this is an expensive operation, this is best left as an actual method.

Key points

- **Methods** are behaviors that extend the functionality of a class.
- A typical method is a function defined inside of a class or object.
- A method can access the value of an instance by using the keyword `this`.
- **Companion object methods** add behavior to a class instead of the instances of that class. To define a companion object method, you add a function in the class `companion object` block.
- You can augment an existing class definition and add methods to it using **extension methods**.

Where to go from here?

Methods and properties are the things that make up your classes, instances, and objects. Learning about them as you have these last two chapters is important since you'll use them all the time in Kotlin.

You've tackled the basics of classes. In the next chapter, you'll learn about some advanced ways to use classes such as inheritance and limiting member visibility.